

Sei un esperto sviluppatore C++. Il tuo compito è scrivere un programma completo, efficiente e ben strutturato che implementi l'analisi di grafi pesati applicata ai cammini di instradamento BGP (Border Gateway Protocol), basandoti sulle specifiche dettagliate descritte di seguito.

1. Suddivisione progetto

Il codice deve essere diviso in tre file:

1. 'grafopesato.h': File di intestazione della classe `GrafoPesato`
2. 'grafopesato.cpp': Implementazione dei metodi della classe `GrafoPesato`
3. 'main.cpp': Programma principale con caricamento dati, misurazione delle prestazioni tramite `std::chrono` e menù interattivo a riga di comando

1.1 Specifiche di **'grafopesato.h'** e **'grafopesato.cpp'**

La classe `GrafoPesato` gestisce un grafo non orientato pesato rappresentato tramite liste di adiacenza con indicizzazione 1-based per i vertici

1.2 Membri Privati

- `int V`: Numero corrente di nodi attivi nel grafo.
- `int E`: Numero corrente di archi nel grafo.
- `static const int MAXN = INT16_MAX`: Costante che limita il numero massimo di nodi ammessi (32767).
- `vector<vector<pair<int, int>>> adj`: Lista di adiacenza. Per ciascun nodo `x`, `adj[x]` contiene coppie `{y, w}` dove `y` è il nodo adiacente e `w` è il peso dell'arco.

1.3 Metodi e Algoritmi da implementare

Costruttori e Metodi Accessori:

- 'GrafoPesato()': Inizializza 'V = 0', 'E = 0' e ridimensiona 'adj' alla dimensione iniziale 1.
- 'GrafoPesato(int v, int e)': Inizializza 'V' ed 'E'. Se 'V > MAXN', stampa un messaggio di errore ed esce dal programma ('exit(1)').
- 'void V_set(int v)' e 'void E_set(int e)': Setter con controllo di validità per 'V' rispetto a 'MAXN'.
- int get_V(), int get_E(), 'vector<vector<pair<int,int>>>' get_adj(): Getter standard per lo stato interno.

Gestione degli Archi

- 'void new_edge(int x, int y, int w)':
- Verifica che gli indici 'x' e 'y' non superino 'MAXN'.
- Ridimensiona 'adj' se necessario fino a 'max(x, y) + 1'.
- Se 'adj[x]' o 'adj[y]' erano precedentemente vuoti, incrementa opportunamente 'V'.
- Inserisce l'arco non orientato (aggiunge '{y, w}' a 'adj[x]' e '{x, w}' a 'adj[y]') e incrementa 'E'.
- 'void add_weight(int x, int y)':
- Cerca l'arco tra 'x' e 'y' in entrambe le liste di adiacenza e incrementa il suo peso di '1' (utilizzato per aggiornare la frequenza d'uso dell'arco nei cammini BGP).

Stampa dell'Adiacenza:

- 'void stampa_adj(ofstream &out)': Scrive su file di testo la lista di adiacenza nel formato 'nodo: (vicino1, peso1) (vicino2, peso2) ...'.

- 'void stampa_adj(vector<int> &unhash, ofstream &out)': Variante che scrive la lista di adiacenza convertendo gli indici interni dei nodi nei loro identificatori originali usando il vettore di rimappatura 'unhash'.

Algoritmi di Cammino e Minimax:

- 'vector<int> costi_minimax(int u)': Calcola il costo Minimax (il cammino che minimizza il peso massimo del singolo arco attraversato, cioè il "bottleneck path") dal nodo 'u' verso tutti gli altri nodi.
- Algoritmo: Inizializza le distanze a 'INF' e 'dist[u] = 0'. Utilizza una priority queue di coppie '{-dist, nodo}'. Per ogni vicino 'y' del nodo corrente 'x' con arco di peso 'w', calcola $M = \max(\text{dist}[x], w)$. Se $M < \text{dist}[y]$, aggiorna $\text{dist}[y] = M$ e reinserisce il nodo nella coda.
- 'int costo(vector<int> &P)': Data una sequenza di nodi rappresentante un cammino P , restituisce il peso massimo tra gli archi consecutivi che lo compongono.

Componente Connessa e Conteggio Cammini:

- 'pair<GrafoPesato, pair<vector<int>, vector<int>>> cc_massima()':
- Trova la componente connessa con il maggior numero di nodi mediante una visita DFS iterativa con 'std::vector' usato come stack.
- Crea un vettore 'id' per rimappare i nodi della componente connessa principale in indici contigui da 1 a V_{CC} (assegnando -1 ai nodi fuori dalla componente).
- Costruisce un nuovo oggetto 'GrafoPesato' T contenente unicamente i nodi ed archi di tale componente re-indicizzati.
- Restituisce la coppia '{T, {id, best}}', dove 'id' è la mappa dai vecchi nodi ai nuovi nodi e 'best' è il vettore inverso.
- 'bool raggiungibile(int u, int v, vector<bool>& visited, int costo)': Funzione ausiliaria di ricerca in profondità con backtracking che verifica se v è raggiungibile da 'u' attraversando solo archi con peso uguale a 'costo'.

- 'int conta_minimax(int u, int v, vector<bool> &visitato, int costo)': Conta ricorsivamente tutti i cammini semplici tra 'u' e 'v' aventi costo massimo per arco 'costo'.
- Ottimizzazione: Prima di esplorare un nodo adiacente, chiama 'raggiungibile' per verificare se esiste un percorso valido da quel nodo fino alla destinazione 'v'. Se non è raggiungibile, pota il ramo di ricorsione.

Ridimensionamento:

'void ridimensiona(int dim)': Tronca il grafo ai primi 'dim' nodi. Rimuove dalla lista di adiacenza tutti gli archi che collegano nodi con indice $i > dim$ e aggiorna di conseguenza 'V' ed 'E'.

2. Specifiche di 'main.cpp'

Il file 'main.cpp' coordina l'I/O dei file, il cronometraggio e un menù utente interattivo.

Flusso di Esecuzione:

Lettura Topologia ('19980101.as-rel.txt'):

- Apre il file di topologia (righe formattate come 'nodoA|nodoB|...', ignorando righe vuote e commenti '#').
- Aggiunge gli archi al grafo 'G' con peso iniziale '0'.

Elaborazione Cammini BGP ('19980101.all-paths'):

- Apre il file dei cammini osservati (ogni riga contiene un prefisso seguito da una sequenza di nodi separati da '|').
- Per ciascun cammino, invoca 'G.add_weight(P[i], P[i+1])' sui nodi consecutivi per incrementare il peso degli archi attraversati.

- Salva i cammini in un vettore 'obs' di coppie '{costo, cammino}' e ne calcola il costo tramite 'G.costo(...)'.

Misura il tempo impiegato per il caricamento tramite 'std::chrono'.

- Esegue 'cc_massima()' su 'G', aggiorna 'G' con la componente gigante e salva le mappe di re-indicizzazione. Misura il tempo dell'operazione.
- Calcola un vettore di frequenze per conteggiare quante volte compare ciascun peso sugli archi del grafo.

Menù Interattivo a Riga di Comando e Scrittura su 'output.txt':

Il programma deve mostrare un menu in un ciclo 'while' che scrive i risultati nel file 'output.txt':

****Opzione 1**:** Stampa il numero di nodi 'V' e archi 'E'. Chiede all'utente se desidera stampare l'intera lista di adiacenza rimappata su file.

****Opzione 2**:** Richiede due nodi 'u' e 'v'. Controlla che appartengano alla componente connessa e scrive su file il costo minimax ottimo calcolato tra di essi.

****Opzione 3**:** Genera su file un istogramma testuale (usando il carattere '/') indicante la distribuzione delle frequenze dei pesi degli archi.

****Opzione 4**:** Richiede due nodi 'u' e 'v'. Calcola il costo minimax ottimo ed esamina il vettore 'obs' per contare quanti cammini BGP reali osservati tra 'u' e 'v' hanno esattamente quel costo minimax ottimo.

****Opzione 5**:** Richiede una dimensione 'dim' per un sottografo e due nodi 'u' e 'v'. Crea una copia del grafo, applica 'ridimensiona(dim)' e conta il numero di cammini minimax ottimi tra 'u' e 'v' usando 'conta_minimax'.

****Opzione 6**:** Stampa i tempi d'esecuzione registrati in millisecondi per le varie fasi del programma.

****Opzione 0**:** Termina il programma.